

PROJECT REPORT

PYTHON-BASED NETWORK SNIFFER

Live Packet Capture and Protocol Analysis

Submitted by	N. Sri Chaithanya
Course / Domain	Cybersecurity
Task Number	01
Submission Date	09-11-2026

Tools: Python · Scapy · Npcap · Flask · HTML/CSS/JavaScript

Abstract

This project implements a basic network sniffer using Python and Scapy to observe IP packets on the local machine's network interface. For each captured IP packet, the program displays source and destination IP addresses, identifies common transport protocols, records port numbers when available, and reports packet size. Where a separate raw payload is present, a short preview is shown for basic inspection.

A lightweight Flask web interface presents recent packets in a browser, with packet counters, protocol filtering, search, and controls to start or stop capture. The application was tested on a Windows computer using locally generated network traffic. This report describes the implementation and observations from the supplied screenshots.

1. Introduction

Network packet inspection helps explain how applications exchange data over a network. A packet sniffer observes traffic and presents selected fields so that a learner can connect protocol concepts with actual network activity. This project focuses on the fundamentals rather than advanced intrusion detection or complete protocol decoding.

2. Objectives

- Capture live IP packets using Python and Scapy.
- Display source and destination IP addresses.
- Identify TCP, UDP, and ICMP packets where present, while categorizing other IP traffic separately.
- Show source and destination ports when available and display packet size.
- Inspect a short payload preview when raw payload bytes are available.
- Provide a local browser interface for viewing and filtering captured packets.
- Generate network traffic and verify that the capture program receives packets.

3. Tools and Technologies

Tool / Technology	Purpose
Python	Core programming language for packet handling and application logic.
Scapy	Captures and inspects packet layers and fields.
Npcap	Provides Windows packet-capture support used by packet-capture libraries.
Flask	Runs the local web server and provides endpoints for the interface.
HTML / CSS / JavaScript	Structures, styles and updates the dashboard.
Windows CMD	Used to install dependencies, run the program and generate test traffic.

4. System Requirements

Category	Requirement
Operating system	Windows
Hardware	Computer or laptop with a network interface
Programming language	Python 3
Python packages	Scapy and Flask
Packet capture support	Npcap, installed and configured
Network	Network connection and permission to inspect relevant traffic
Browser	Modern web browser
Execution permissions	Appropriate permissions for packet capture

5. Implementation

5.1 Packet Capture and Inspection

The Scapy capture routine listens for packets and passes each captured packet to a handler. The handler checks for an IP layer and extracts the source and destination IP addresses. It then checks for TCP, UDP, or ICMP layers.

TCP and UDP source and destination ports are displayed when available. Packet length is used as the packet-size value. If a Raw layer exists, the command-line version displays a limited text preview and a hexadecimal representation of the payload bytes.

5.2 Local Web Interface

The Flask application serves the dashboard at 127.0.0.1 on port 5000. The browser periodically requests the recent packet list from the local application. The dashboard displays packet totals, TCP/UDP/ICMP counters, capture status, and a table containing time, IP addresses, protocol, ports, packet size, and payload preview. Search and protocol filtering help narrow the visible list.

The application retains up to 500 recent packet records in memory.

6. Test Traffic and Results

A basic HTTPS request was generated from Windows CMD using the following command:

```
curl.exe --http1.1 https://example.com/ -o NUL
```

The screenshot shows that the curl command completed and received 577 bytes from example.com. This confirms the request succeeded. The live dashboard and command-line packet output provide separate evidence that the sniffer was receiving network packets during testing.

The supplied dashboard screenshot shows the capture status as “Capturing” and 500 recent packet records. At the time of that screenshot, the counters showed 202 TCP packets, 228 UDP packets, and 0 ICMP packets. These counters represent the protocol composition of the stored 500 records at that moment, not the total number of packets ever seen.

The table includes local traffic such as packets from 192.168.29.101 to 192.168.29.255 and UDP multicast traffic to 239.255.255.250 on port 1900. A payload preview includes an SSDP discovery message, illustrating that some UDP application data can be readable.

7. Screenshots and Evidence

Figure 1: HTTPS test request and project files.

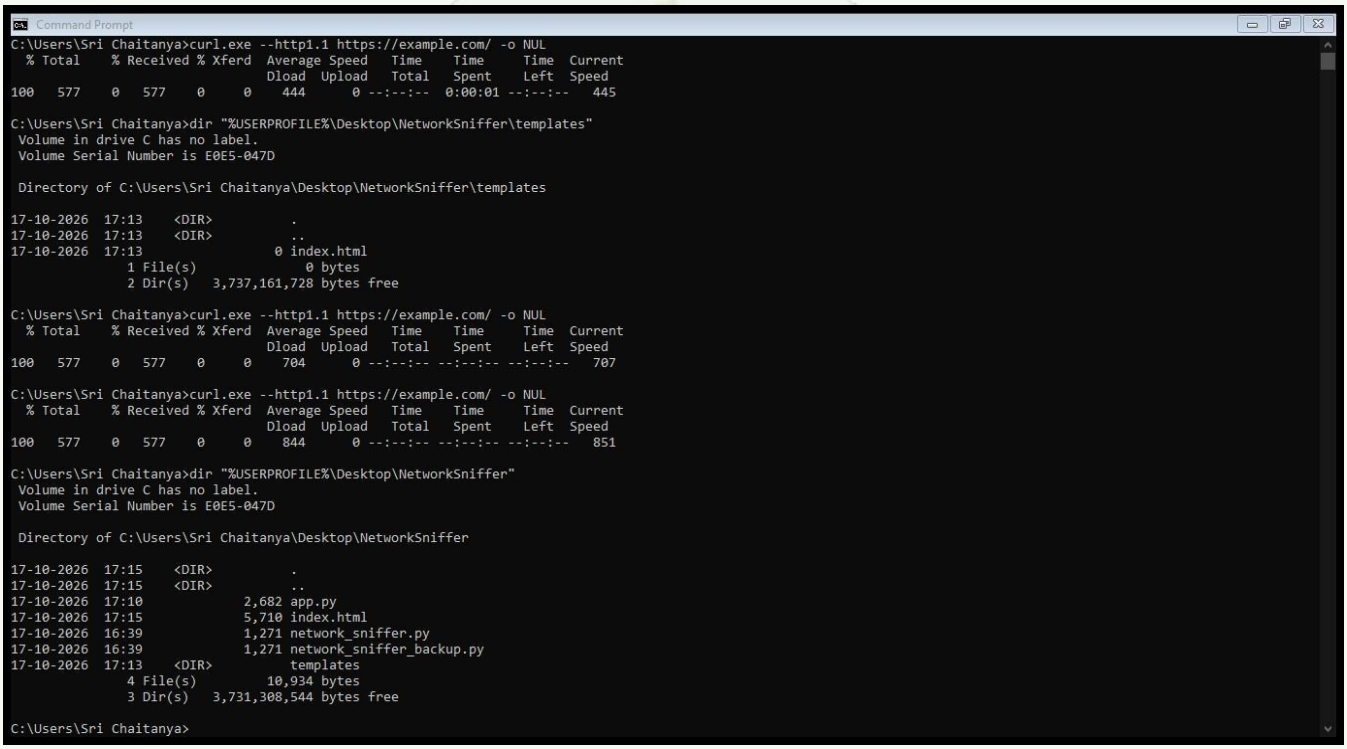


Figure 2. Command-line packet inspection output.

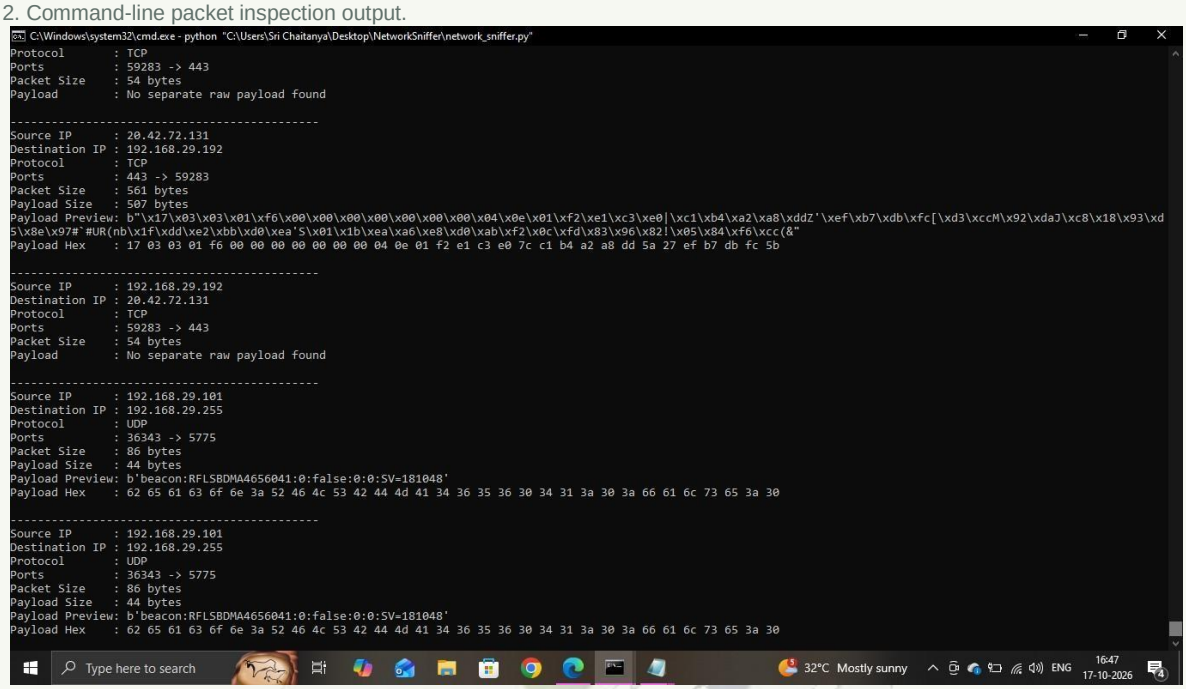
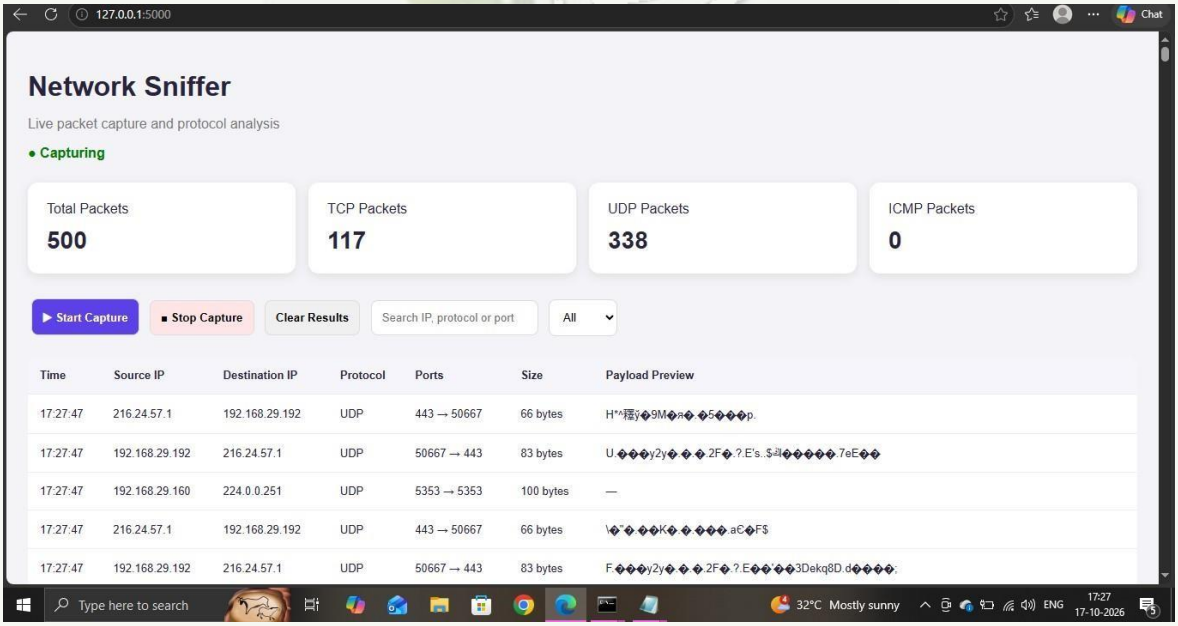


Figure 3 : Browser based network sniffer dashboard (running in local host server)



8. Testing

Test case	Expected result	Observed result
Start Flask application	Local server starts on port 5000.	Server log showed it running at 127.0.0.1:5000.
Open dashboard	Dashboard loads in the browser.	Dashboard displayed capture status, counters and packet table.
Generate HTTPS request	curl completes successfully.	577 bytes received from example.com.
Observe live traffic	Packet fields are displayed.	IP addresses, protocols, ports, sizes and payload previews appeared.
Search and filter packets	Visible packet list can be narrowed.	Controls are present in the dashboard; functionality should be verified before claiming the test passed.

9. Limitations

- The dashboard stores only the latest 500 packet records in memory; records are not automatically saved to disk.
- The capture logic focuses on IP packets and basic fields rather than complete decoding of every protocol.
- Encrypted application traffic generally cannot be interpreted as readable plaintext from a simple payload preview.
- Packet counts represent a snapshot of currently stored records and can change while capture continues.
- The Flask development server is intended for local testing, not public production deployment.
- On Windows, packet capture may require Npcap to be installed correctly and the terminal to run with suitablepermissions.

10. Safety and Responsible Use

The sniffer should be used only on the user's own computer, network, or environments where explicit permission has been granted. Captured packets may contain private information and should be handled securely. The local dashboard is bound to 127.0.0.1, which limits normal access to the local machine.

11. Future Enhancements

Enhancement	Description
CSV Export	Export packet records for further analysis in spreadsheet tools.
PCAP Export	Save packet captures for deeper analysis in Wireshark.
DNS Analysis	Add DNS-specific parsing and readable query information.

Error Handling	Display clear errors for capture permissions and interface selection.
Packet-Rate Summary	Add timestamp-based packet-rate statistics.
Persistent Storage	Store selected packet records for later review.

12. What I Learned

Through this project, I learned what network packets are and how a network sniffer captures packets moving across a network interface. I learned to examine packet structure and available content, including source and destination IP addresses, protocols, port numbers, packet size, and a short payload preview.

I learned the basic differences between TCP and UDP traffic and how protocol and port information can help explain communication between devices and applications. I gained practical experience using Python and Scapy for packet capture, and Flask, HTML, CSS, and JavaScript for presenting captured packet information through a local browser dashboard.

A payload preview does not always reveal complete application content: encrypted or binary traffic may appear as unreadable bytes. Packet observations should be interpreted carefully without assuming every captured packet is suspicious.

Appendix A — GitHub README

The following README content is also provided as a separate Markdown file for the GitHub repository.

Project Overview

A Python-based network sniffer that observes IP packets on a local network interface using Scapy and presents selected packet details through command-line output and a local Flask dashboard.

Objectives

- Capture IP packets using Python and Scapy.
- Display source/destination IP addresses and identify TCP, UDP, ICMP and other IP traffic.
- Show ports when available and packet size.
- Present recent packet records in a local browser dashboard.
- Generate test traffic and observe packet capture.

Tools and Requirements

Python 3; Scapy; Npcap for Windows packet capture; Flask; HTML/CSS/JavaScript; Windows; network interface; modern browser; appropriate capture permissions.

How It Works

1. Scapy listens for packets.
2. The handler checks the IP layer and extracts addresses.
3. TCP, UDP and ICMP layers are identified where present.
4. TCP/UDP ports and packet size are shown when available.
5. Flask serves the dashboard at `http://127.0.0.1:5000`.
6. The browser requests recent packet records from the local app.

```
# Horizon TechX — Task 01: Python-Based Network Sniffer

## Project overview

This project demonstrates a Python-based network sniffer for observing IP
packets visible to the local computer's network interface. It uses Scapy for
packet capture and inspection, and a local Flask web dashboard to present
recently captured packet information.

The project was developed as a practical cybersecurity learning exercise to
understand packet structure, common network protocols, and basic traffic
observation.

## Objectives

- Capture IP packets using Python and Scapy.
- Display source and destination IP addresses.
- Identify TCP, UDP, ICMP, and other IP traffic.
- Display TCP/UDP port numbers when available.
- Show packet size and a short payload preview when raw payload data is
  available.
- Present recent packet information through a local browser dashboard.
- Generate test traffic and observe the packets received by the sniffer.

## Tools and technologies

| Tool / technology | Purpose |
|---|---|
| Python 3 | Main programming language |
| Scapy | Packet capture and packet-layer inspection |
| Npcap | Windows packet-capture support |
| Flask | Local web server and dashboard endpoints |
```

| HTML, CSS, JavaScript | Dashboard structure, styling, and browser updates |
| Windows Command Prompt | Installing dependencies, running the application, and generating test traffic |

Project workflow

1. The packet-capture routine listens for traffic visible on the selected or available network interface.
2. When an IP packet is received, the packet handler extracts its source and destination IP addresses.
3. The handler checks for TCP, UDP, and ICMP layers. Other IP traffic is categorized separately.
4. TCP and UDP port numbers are displayed when available.
5. Packet length is used to report packet size. A short payload preview can be shown when a raw payload is present.
6. The Flask application provides a local dashboard at `http://127.0.0.1:5000`.
7. The browser periodically requests the recent packet records and displays them for inspection.

Testing and observations

The project report documents the following test activity and observations:

- A test HTTPS request was generated from Windows Command Prompt using:

```cmd
curl.exe --http1.1 https://example.com/ -o NUL
```

- The command output recorded a 577-byte response from `example.com`. - The supplied dashboard screenshot showed the capture status as **Capturing** and 500 recent packet records.
- At the time of that screenshot, the stored records included 202 TCP packets, 228 UDP packets, and 0 ICMP packets.
- The command-line evidence showed packet fields such as source and destination addresses, protocol, ports, packet size, and payload preview.

The packet counts above are a snapshot of the most recent stored records shown in the report. They are not lifetime totals. The successful HTTPS request confirms that the request completed; the dashboard and command-line screenshots provide the separate packet-capture evidence.

Limitations

- The dashboard retains up to 500 recent packet records in memory. - Packet records are not automatically saved to disk.
- The capture focuses on IP packets and basic packet fields rather than complete decoding of every protocol.
- Encrypted application traffic generally cannot be interpreted as readable plaintext through a basic payload preview.
- The Flask development server is intended for local testing, not public production deployment.
- Search and protocol-filter controls should only be described as verified after their behaviour has been tested.

Responsible use

Use this project only on your own computer or network, or in an environment where you have explicit permission to inspect traffic. Captured packets may contain private information. Do not publish raw packet captures, payloads, credentials, or other sensitive network evidence.

The dashboard is configured for local access through `127.0.0.1`.

Repository contents for this submission

This submission contains the project documentation:

```text
Task-1_Network-Sniffer/
├── README.md
└── Horizon_TechX_Network_Sniffer_Report.pdf
```

The report contains the project explanation, implementation overview, test observations, screenshots, limitations, and learning outcomes. This repository submission does not include the application's source-code files.

Learning outcomes

Through this project, I gained practical understanding of:

- How IP packets are observed during network communication.
- The basic differences between TCP, UDP, and ICMP traffic.
- How source/destination addresses, ports, packet sizes, and payload previews help describe packets.
- Using Python and Scapy for packet capture and inspection.
- Using Flask with HTML, CSS, and JavaScript to present packet information in a local browser dashboard.

```
- Interpreting packet observations carefully without assuming that every
  captured packet is suspicious.

## Future enhancements

Possible future improvements include CSV and PCAP export, DNS-specific
parsing, clearer capture-permission and interface-selection errors, packet
rate summaries, and persistent storage for selected packet records.
```

Limitations and Safety

The dashboard retains up to 500 records in memory and does not automatically persist them. It focuses on basic IP fields; encrypted traffic is not generally readable as plaintext. Use only on networks/devices you own or are authorized to monitor. Treat captured data as private.

13. Conclusion

The project demonstrates the basic workflow of capturing network packets, extracting key fields, and presenting the results through both command-line output and a local web dashboard. The observed traffic and successful test request show that the application can serve as a practical learning tool for packet structure and common network protocols. It provides a foundation for future packet export, protocol-specific parsing, and deeper network analysis.

